

ESP32 1 — ESP32 Setup and First Circuit

Tutorial Group: ESP32

Goal

In this tutorial, you will configure the Arduino IDE to program an ESP32-WROOM-32 and build a simple ESP32 circuit. The ESP32 software support is an addition to the Arduino IDE installation used in the Arduino tutorials; it is not a separate IDE.

This tutorial assumes that you have already completed *Arduino 1 — Install the Arduino IDE and Run the Blink Test*, including installation and basic use of the Arduino IDE. You should also complete the Arduino tutorials before beginning the ESP32 series.

What You Need

- ESP32-WROOM-32
- USB cable appropriate for the ESP32-WROOM-32
- Computer with the Arduino IDE installed
- Breadboard and jumper wires
- LED and 220 Ω resistor from the Mega 2560 evaluation kit

Part 1 — Understand the Two Microcontroller Platforms

The competition tutorials use two different microcontroller platforms: the Arduino Mega 2560 and the ESP32-WROOM-32. Both can be programmed from the Arduino IDE, but they are different processors with different hardware capabilities, pin assignments, and electrical characteristics.

The Arduino IDE can support both processors. You install the IDE once, then add ESP32 board support to it. You do not need a second copy of the Arduino IDE just because you are programming an ESP32.

Part 2 — Add ESP32 Board Support to the Arduino IDE

The ESP32 board definitions are installed through the Arduino IDE's Board Manager. This allows the IDE to understand how to compile and upload programs for ESP32 boards.

Open the Arduino IDE and locate the Preferences/Settings area. Find the field for Additional Boards Manager URLs.

Use the official Espressif Arduino-ESP32 Boards Manager package URL: https://espressif.github.io/arduino-esp32/package_esp32_index.json

After saving the settings, open the Boards Manager and search for ESP32. Install the ESP32 board package provided by Espressif Systems.

The exact names and locations of these menu items can vary between Arduino IDE versions. If your IDE looks different, use the IDE's documentation or the current official Espressif Arduino-ESP32 installation instructions rather than relying on an older URL or third-party guide.

Part 3 — Select the ESP32 Board

After the ESP32 board package has been installed, connect the ESP32 board to your computer with USB.

Use the Arduino IDE's board-selection controls to select the ESP32 board that matches your hardware. For a typical ESP32-WROOM-32, the board definition may be listed as an ESP32 Dev Module or another board name corresponding to the specific development board.

Select the serial port associated with the ESP32. The available port name will depend on your computer and the USB interface on the particular board.

If the ESP32 does not appear as a selectable port, check the USB cable, USB connection, and the board's USB-to-serial driver requirements.

Part 4 — One Laptop or Two?

Because the same Arduino IDE can program both the Arduino Mega 2560 and the ESP32, a team can use one laptop for both.

If using one laptop, the important rule is: always verify the selected board and port before compiling and uploading.

- For Arduino work, select the Arduino Mega 2560 and its corresponding port.
- For ESP32 work, select the appropriate ESP32 board and its corresponding port.
- Do not assume that the IDE is still configured for the processor you worked with previously.
- The selected board determines how the program is compiled, so selecting the wrong board can produce errors or result in a program being sent to the wrong device.

If the team has two laptops available, it may be easier to dedicate one laptop to the Arduino Mega 2560 and the other to the ESP32. This avoids repeatedly changing the selected board and port and can be particularly convenient when both processors are being used during the competition.

Part 5 — Build a Simple ESP32 Circuit

Build a simple LED circuit using the ESP32. Use an LED and a 220 Ω resistor, just as you did in the Arduino tutorials.

Do not assume that the Arduino pin numbers from the previous tutorials apply to the ESP32. ESP32 GPIO numbers are different from Arduino Mega 2560 pin numbers, and the ESP32 has additional restrictions on some GPIOs.

Choose a suitable ESP32 GPIO for the LED and record the GPIO number you selected. Connect the LED through the 220 Ω resistor to the selected GPIO and connect the other side of the LED to GND.

Part 6 — Write a Simple Blink Program

Create a new Arduino sketch for the ESP32. Use the same basic program structure you learned in the Arduino tutorials:

```
const int ledPin = /* your ESP32 GPIO number */;

void setup()
{
    pinMode(ledPin, OUTPUT);
}

void loop()
{
    digitalWrite(ledPin, HIGH);
    delay(1000);

    digitalWrite(ledPin, LOW);
    delay(1000);
}
```

For this tutorial, use GPIO 25 for the LED. If your ESP32-WROOM-32 board labels or exposes the pins differently, check its documentation for a suitable GPIO pin.

For this first ESP32 tutorial, using `delay()` is intentional. The purpose here is to verify that you can compile, upload, and run a program on the ESP32. You already learned the advantages of `millis()`-based timing in *Arduino 4 — Non-Blocking Timing with millis()*.

Part 7 — Compile and Upload

Compile the program and then upload it to the ESP32.

If the upload does not begin normally, some ESP32 development boards require the BOOT button to be pressed during the beginning of the upload process. Follow the behavior of your particular board and the messages displayed by the Arduino IDE.

Once the program has uploaded successfully, the LED should turn on for approximately one second and off for approximately one second.

Part 8 — Compare the Arduino and ESP32

The programming concepts are familiar because the ESP32 can be programmed through the Arduino environment. However, the underlying hardware is different.

- The Arduino Mega 2560 and ESP32 use different microcontrollers.
- Their GPIO numbers and available pins are different.
- Their analog inputs are different and do not use the same ADC numbering scheme.
- The ESP32 generally operates at 3.30v logic rather than the 5.00v logic used by the Arduino Mega 2560.
- Some ESP32 GPIOs have special functions or restrictions and should not automatically be treated as general-purpose I/O.
- The ESP32 provides significantly greater processing capability and additional peripherals, including built-in Wi-Fi and Bluetooth.

WARNING: ESP32 GPIO inputs MUST NOT be connected to signals greater than 3.30v. Before connecting ANY external circuit to an ESP32 GPIO, verify that the voltage on that signal cannot exceed 3.30v. A 5.00v signal can damage the ESP32. Do not assume that a circuit is safe simply because it worked on the Arduino Mega 2560.

These differences become important when moving a circuit or program from one platform to the other. Do not assume that a circuit that works on the Arduino can simply be moved to the same-numbered ESP32 pin.

Part 9 — Repeat the Earlier Programming Tutorials

The remaining Arduino tutorials do not need to be rewritten here as separate ESP32 tutorials. Instead, repeat the programming tutorials from the Arduino tutorials on the ESP32.

In particular, use the ESP32 to practice the same basic concepts you learned with the Arduino:

- Digital output and LED control.
- Reading an analog input from a variable resistor.
- Converting the ADC reading to a voltage.
- Printing values to the Serial Monitor.
- Using `millis()` for non-blocking timing.
- Controlling multiple independently timed activities.

When repeating these tutorials, identify the ESP32 GPIO or ADC pin that replaces the Arduino pin used in the original tutorial. Consult the documentation for your specific ESP32 development board rather than relying on Arduino Mega 2560 pin numbers.

Important ESP32 Habit

Before connecting a component to an ESP32 GPIO, determine whether that GPIO is appropriate for the intended use. The ESP32 has more pin-specific behavior than the Arduino Mega 2560, and some pins should be avoided for particular applications.

Develop the habit of checking the board documentation before choosing a pin. This will be especially important in the competition, where understanding the hardware rather than simply copying an Arduino tutorial may be necessary.

What You Should Understand

- The ESP32 can be programmed using the Arduino IDE.
- ESP32 support is added to an existing Arduino IDE installation.

- The Arduino Mega 2560 and ESP32 can be programmed from the same IDE.
- When using one laptop, the correct board and port must be selected before working with either processor.
- Using two laptops allows a team to dedicate one computer to each processor.
- ESP32 GPIO numbers are different from Arduino Mega 2560 pin numbers.
- The ESP32 uses 3.30v logic and has GPIO-specific restrictions.
- The Arduino programming concepts learned in the earlier tutorials can be transferred to the ESP32, but the hardware details must be checked.

Before Moving On

You should be able to answer **yes** to all of the following:

- ☐ I have completed *Arduino 1 — Install the Arduino IDE and Run the Blink Test* and have the Arduino IDE working.
- ☐ I have installed ESP32 board support in the Arduino IDE.
- ☐ I can select the ESP32 board and its serial port.
- ☐ I can switch the IDE between the Arduino Mega 2560 and ESP32.
- ☐ I can identify the GPIO number I am using for an LED.
- ☐ I can compile and upload a program to the ESP32.
- ☐ I understand that ESP32 GPIO numbers cannot be assumed to match Arduino Mega 2560 pin numbers.
- ☐ I understand that the ESP32 uses 3.30v logic.
- ☐ I can repeat the basic Arduino programming tutorials on the ESP32 using appropriate ESP32 pins.

Once this tutorial is complete, your team should be comfortable moving between the Arduino Mega 2560 and ESP32 development environments.